- ***Code*** — The region where the program instructions are stored.
- ***Static memory*** — The region where global variables (variables declared outside any function) as well as static local variables (variables declared inside functions starting with the keyword "static") are allocated. Static variables are allocated once and stay in the same memory location for the duration of a program's execution.
- ***The stack*** — The region where a function's local variables are allocated during a function call. A function call adds local variables to the stack, and a return removes them, like adding and removing dishes from a pile; hence the term "stack." Because this memory is automatically allocated and deallocated, it is also called ***automatic memory***.
- ***The heap*** — The region where the "new" operator allocates memory, and where the "delete" operator deallocates memory. The region is also called ***free store***.
- A ***try*** block surrounds normal code, which is exited immediately if a throw statement executes.
- A ***throw*** statement appears within a try block; if reached, execution jumps immediately to the end of the try block. The code is written so only error situations lead to reaching a throw. The throw statement provides an object of a particular type, such as an object of type "runtime_error", which is a class defined in the ***stdexcept library***. The statement is said to throw an exception of the particular type. A throw statement's syntax is similar to a return statement.
- A ***catch*** clause immediately follows a try block; if the catch was reached due to an exception thrown of the catch clause's parameter type, the clause executes. The clause is said to catch the thrown exception. A catch block is called a ***handler*** because it handles an exception.

Type	Reason exception is thrown
bad_alloc	Failure in allocating memory
ios_base::failure	Failure in a stream (Ex: cin, stringstream, fstream)
logic_error	To report errors in a program's logic. Ex: out_of_range error (index out of bounds)
runtime_error	To report errors that can only be detected at runtime. Ex: overflow_error (arithmetic overflow

```
int main() {

   int expMonth;
```

```
  int expYear;


  cin >> expMonth;
  cin >> expYear;


  try {
    if ((expMonth < 1) || (expMonth > 12)) {
      throw runtime_error("Month must be between 1 and 12");
    }
    if (expYear < 2024) {
      throw runtime_error("Year must be at least 2024");
    }
    cout << "Passport expires on " << expMonth << "/" << expYear << endl;
  }
  catch (runtime_error& excpt) {
    cout << "Error: " << excpt.what() << endl;
  }
  return 0;
}
```

The power of exceptions becomes clearer when used within a function. If an exception is thrown within a function and not caught within that function, then the function is immediately exited and the calling function is checked for a handler, and so on up the function call hierarchy. The following illustrates; note the clarity of the normal code.

```
#include <iostream>
#include <stdexcept>
using namespace std;

int GetWeight() {
   int weightParam;     // User defined weight

   // Get user data
   cout << "Enter weight (in pounds): ";
   cin >> weightParam;

   // Error checking, non-negative weight
   if (weightParam < 0) {
      throw runtime_error("Invalid weight.");
   }
   return weightParam;
}

int GetHeight() {
   int heightParam;     // User defined height

   // Get user data
   cout << "Enter height (in inches): ";
   cin >> heightParam;

   // Error checking, non-negative height
   if (heightParam < 0) {
      throw runtime_error("Invalid height.");
   }
   return heightParam;
}

int main() {
   int weightVal;        // User defined weight
(lbs)
   int heightVal;        // User defined height
(in)
   float bmiCalc;        // Resulting BMI
   char quitCmd;         // Indicates quit/continue

   quitCmd = 'a';

   while (quitCmd != 'q') {
      try {
         // Get user data
         weightVal = GetWeight();
```

```
Enter weight (in
pounds): 150
Enter height (in
inches): 66
BMI: 24.208
(CDC: 18.6-24.9
normal)

Enter any key ('q' to
quit): a
Enter weight (in
pounds): -1
Invalid weight.
Cannot compute health
info.

Enter any key ('q' to
quit): a
Enter weight (in
pounds): 150
Enter height (in
inches): -1
Invalid height.
Cannot compute health
info.

Enter any key ('q' to
quit): q
```

```
         heightVal = GetHeight();

         // Calculate BMI and print user health
info if no input error
         // Source: http://www.cdc.gov/
         bmiCalc = (static_cast<float>(weightVal) /
                    static_cast<float>(heightVal *
heightVal)) * 703.0;

         cout << "BMI: " << bmiCalc << endl;
         cout << "(CDC: 18.6-24.9 normal)" << endl;
      }
      catch (runtime_error &excpt) {
         // Prints the error message passed by
throw statement
         cout << excpt.what() << endl;
         cout << "Cannot compute health info." <<
endl;
      }

      // Prompt user to continue/quit
      cout << endl << "Enter any key ('q' to quit):
";
      cin >> quitCmd;
   }

   return 0;
}
```

```
#ifndef ARRAY_H

#define ARRAY_H


#include <string>

#include <sstream>

#include <stdexcept>

using namespace std;
```

```
/* class: Array
 * The class template that represents an array of
 * any type elements.
class Array {
public:
    int *items = nullptr;
    int size = 0;
    Array(const int size) {
        if (size < 0) {
            throw range_error("Invalid size: " + to_string(size));
        }
        this->size = size;
        items = new int[size]{};
    }
    ~Array() {
        delete[] items;
        size = 0;
    }
    int getElementAt(const int index) {
        if (index < 0 || index >= size) {
            throw range_error("Index out of bounds: " + to_string(index));
        }
```

```
    return items[index];
}
void setElementAt(const int index, const int value) {
    if (index < 0 || index >= size) {
        throw range_error("Index out of bounds: " + to_string(index));
    }
    items[index] = value;
}
string descriptor() const {
    ostringstream oss;
    if (size == 0) {
        oss << "[]";
    } else {
        oss << "[";
        for (int i = 0; i < size; ++i) {
            oss << items[i];
            if (i < size - 1) {
                oss << ", ";
            }
        }
        oss << "]";
    }
    return oss.str();
```

```
    }
};


#endif
```

- ★ 8/26 Lecture
    - Skill: a thing you learn or are taught. (Right now, it's kinda poor)
    - Technique: applying a skill. (I might have it a little, but…)
    - Style: a way you apply skills (I had a style… with C. This ain't C)
    - Habit: the way I do things after some time that's hard to get rid of
    - Conventions: the thing that programmers agree to do the same way
    - Bad decisions lead to learning…
    - Constants = all caps (???). They're variables with fixed values that cannot be changed.
    - When you don't name a constant in all caps, you're violating a convention and committing a war crime.
    - One of the most hopeless things is writing something and no one uses it when you could use conventions.
    - By convention, people don't use global variables (at least in this class).
    - Follow conventions laid out by others. But isn't it cheating?
    - When you're moving through the course, you'll level up your skills and techniques.
    - Things are abstracted so they are easier to deal with procedurally.
    - Encapsulation is an object-oriented term.
    - class Circle {
        - private: ← encapsulation (from public to private)
            - double radius;
    - } ;
    - Information hiding: use the minimum visible scope for every variable.
    - int addThreeInts(int a, int b, int c) {
        - return a + b + c;
    - }

Programs need a way to output data to a screen, file, or elsewhere. An ***ostream***, short for "output stream," is a class that supports output, available via `#include <iostream>` and in namespace "std". ostream provides the ***<< operator***, known as the ***insertion operator***, for converting different types of data into a sequence of characters. That sequence is normally placed into a buffer, and the system then outputs the buffer at various times. The << operator returns a reference to the ostream that called the operator, and is evaluated from left to right like most operators, so << operators can appear in series.

The << operator is overloaded with functions to support the various standard data types, such as int, bool, float, etc., each function converting that data type to a sequence of characters. The operator may be further overloaded by the string library from `#include <string>` or by the programmer for programmer-created classes.

cout is a predefined ostream object (declared as `ostream cout;` in the iostream library) that is pre-associated with a system's standard output, usually a computer screen.

Programs need to receive input data, whether from a keyboard, touchscreen, or elsewhere. An ***istream***, short for "input stream," is a class that supports input. Available via `#include <iostream>`, istream provides the ***>> operator***, known as the ***extraction operator***, to extract data from a data buffer and write the data into different types of variables.

cin is a predefined istream pre-associated with a system's standard input, usually a computer keyboard. The system automatically puts the standard input into a data buffer associated with cin. The >> operator skips leading whitespace and extracts as many characters as possible consistent with the target variable's type. The operator then stops at the next whitespace, converts the extracted characters to the target variable's type, and stores the result into the variable.

Function Overloading:

```
void fun () {
…

}
```

^^ Gets called by fun( ). Variables inside are local variables, and <u>fun ()</u> is its own signature.

void fun(int x) {

…

}

^^ Gets called by fun(4)

int x;

Variable x has 4 identifiable parts.

1. Is a name (identifier)
2. Of a location in RAM (address)
3. That stores a value (in binary)
4. Of a type (int, char, double, …)

fixed	Use fixed-point notation. From <iostream>	`// 12.340000` `cout << fixed << 12.34;`
scientific	Use scientific notation. From <iostream>	`// 1.234000e+01` `cout << scientific << 12.34;`

setprecision(p)	If stream has not been manipulated to fixed or scientific:
	Sets max number of digits in number
		`// 12.3`
		`cout << setprecision(3) << 12.34;`
		
		`// 12.34`
		`cout << setprecision(5) << 12.34;`

	If stream has been manipulated to fixed or scientific:
	Sets max number of digits in fraction only (after the decimal point).
	From <iomanip>
		`// 12.3`
		`cout << fixed << setprecision(1) << 12.34;`
		
		`// 1.2e+01`
		`cout << scientific << setprecision(1) << 12.34;`

showpoint	Even if fraction is 0, show decimal point and trailing 0s.
	Opposite is noshowpoint.
	From <iostream>
		`// 99`
		`cout << setprecision(3) << 99.0;`
		
		`// 99.0`
		`cout << setprecision(3) << showpoint << 99.0;`

Prog.exe

Virtual Address Space

0000

x (whatever address it has) ((This is a stack frame))
x ( ) y ( ) ((Functions being called will have their own stack frames))

Dynamic Heap …
Static Heap
Code

1111

```
int main ( ) {
        int x;
}
int fun ( int x ) {
        double y;
}
```

After a function is done, its stack frame ends, is deallocated and disappears. The stack frames together as functions run is called the run-time stack that constantly changes depending on what's happening in the program. It has a limit to it, and when it reaches its limit, it becomes stackOverflow. The space between the static heap and the run-time/beginning stack frames is the dynamic heap.

I'm not ready to code yet until I learn? Then why the heck are the assignments on ZyBooks still due?? We're currently learning what we should have learned the first week but didn't because bro was sick!